

Языки. Страна. Локаль

Кодовое обозначение локали

Чтобы различать локали, принято использовать следующие кодовые обозначения:

1 [язык] – [СТРАНА]

- [язык] — двухбуквенный код языка по стандарту [ISO 639-1](#).
- [СТРАНА] — двухбуквенный код страны по стандарту [ISO 3166-1](#).

Например, ru-RU (Русский язык в России) или ru-KZ (Русский язык в Казахстане).

Один язык в нескольких странах

Один язык может использоваться в нескольких странах, при этом перевод одной и той же фразы может отличаться. Также имеются различия в форматах дат и валют. Например, для английского языка в Британии и США:

en-GB	en-US	ru-RU
Upload cancelled .	Upload canceled .	Загрузка отменена.
Invitation to a dialogue .	Invitation to a dialog .	Приглашение к диалогу.
Due date 31.01.20	Due date 01.31.20	Срок до 31 января 2020

Несколько языков внутри одной страны

Wasaby Framework позволяет поддерживать несколько языков внутри одной страны. Например, для России можно создать словарь как на русском, так и на татарском языке. Такой функционал даёт преимущество Wasaby Framework в том, что можно создавать продукт с возможностью выбора локализации web-приложения на негосударственный язык (язык меньшинства).

Язык ключей перевода

Не имеет значения какой язык используется в исходных файлах. Строки могут быть как на русском, так и на украинском или китайском языках. Особенность Wasaby Framework в том, что принято использовать ключи на Русском языке, это увеличивает читаемость кода, подробнее про это можно прочитать в [статье](#).

Изменение локали с помощью меню выбора языка

Для изменения текущего языка web-приложения, как правило, используют меню выбора языка. Для крупных web-приложений, например [online.sbis.ru](#), оно уже создано. Такое меню выполняет, как правило, два действия:

1. Устанавливает cookie с названием "lang".
2. Устанавливает язык в настройках профиля пользователя.
3. Перезагружает web-страницу.



Диспетчер распространяет язык с помощью cookie "lang" на все приложения, в которых авторизован или зарегистрирован пользователь. Задержка при смене языка может достигать двух минут.

Ниже приведён пример кода, изменяющего локаль в cookie:

```

1 import { controller } from 'I18n/i18n';
2
3 class MyLangSelector {
4     setEnglishLang(): void {
5         // Устанавливаем английский язык.
6         controller.setLang('en');
7
8         // Перезагружаем страницу
9         window.location.reload();
10    }
11 }
12
13 export default MyLangSelector;

```

Механизм получения языка

В приложениях СБИС выбор языка основывается на настройках профиля авторизованного пользователя. В зависимости от типа приложения механизмы получения языка могут варьироваться.

Глобально можно выделить две стратегии получения языка:

- Из настроек профиля авторизованного пользователя.
- Из настроек системы, если нет доступа к профилю.

Приложения без аутентификации пользователей используют язык системы. Например, showcase, отправщик SMS.

Для web-приложений:

- Из настроек профиля авторизованного пользователя, через провайдер языка посредством транспорта cookie lang.
- Из настроек браузера, если нет доступа к профилю.

Для offline-приложений существует две базовых стратегии получения языка:

- Из настроек профиля авторизованного пользователя через провайдер языка.
- Из настроек ОС, если нет доступа к профилю.

Для приложения SBIS3 Plugin существует специфичный сценарий. В приложении одновременно может быть авторизовано несколько пользователей, которые принадлежат разным профилям. В таком случае настройки языка задаются настройками приложения.

Реализация провайдера языка

Для создания собственного провайдера языка необходимо реализовать интерфейс [IAppLanguageProvider](#) и зарегистрировать его в фабрике обработчиков.

1. Создайте класс-наследник IAppLanguageProvider и переопределите у него метод ResolveCurrentLanguage().

Пример реализации провайдера, который возвращает русский язык:

```

1 class MyProvider : public IAppLanguageProvider
2 {
3 private:
4     String ResolveCurrentLanguage() const noexcept override
5     {
6         return L"ru-RU";
7     }
8 };

```

2. Зарегистрируйте в фабрике обработчиков. Пример регистрации обработчика языка:

```

1 Factory::TemplateItem< IAppLanguageProvider, MyProvider > LANG (
    GetLanguageProviderFactory() );

```

Определение языка по настройкам браузера без доступа к профилю пользователя

Предпочитаемый язык определяется по заголовку "Accept-Language". Если заголовок не задан, то языком страницы объявляется Английский — 'en'.

Формальное описание грамматики заголовка:

```
1 Accept-Language = "Accept-Language" ":"
2                  1#( language-range [ ";" "q" "=" qvalue ] )
3     language-range = ( ( 1 * 8ALPHA *( "-" 1 * 8ALPHA ) ) | "*" )
4     qvalue         = ( "0" [ "." 0 * 3DIGIT ] )
5                   | ( "1" [ "." 0 * 3("0") ] )
```

Подробнее о заголовке читайте здесь:

- [2 Notational Conventions and Generic Grammar. RFC 2616](#)
- [14.4 Accept-Language. RFC 2616](#)

Пример: Accept-Language: da, en-GB; q = 0.8, fr; q = 0.7

В данном примере указано, что пользователь предпочитает датский, но готов принять британский английский и любой тип французского.

Рассмотрим внимательно синтаксис этого выражения:

```
1 <Заголовок>: <предпочитаемый язык>, <альтернативный язык>; <коэффициент
    предпочтительности q>
```

Для предпочитаемого языка коэффициент не указывается — что $q=1$.

Алгоритм выбора языка

1. Разделяем переданный заголовок по "весам".
2. Сортируем полученные наборы в порядке уменьшения "весов".
3. Обходим полученные наборы языков в порядке уменьшения "весов".
4. Для каждого языка ищем соответствующий ему [словарь](#):
 - Если словарь есть, то этот язык объявляем "текущим", завершаем цикл и возвращаем его.
 - Если словаря нет, язык задан с уточнением региона и "наиболее подходящий" язык не определен.
 - Ищем словарь без учета региона. Если нашли словарь, то сохраняем его как "наиболее подходящий".
 - Иначе возвращаем 'en' — английский.

Пример реализации алгоритма для выбора предпочитаемого языка

```
1 langs = parse( headers['Accept-Language'] )
2 prefer = None
3 for lang in langs:
4     if dict_exists( lang ):
5         return lang # Ищем точное совпадение словаря и переданного
        языка.
6     delim = lang.find('-')
7     if delim != -1 and prefer is None and dict_exists( lang[ :delim ]
8 ):
9         prefer = lang[ :delim ] # Наиболее подходящим языком объявляем
        язык с наибольшим весом без учета региона.
9 if prefer:
10     return prefer
```

```
11 else:
12     return 'en'
```

Получение текущего языка

```
1 // TypeScript
2 import {controller} from 'I18n/i18n';
3
4 export default function Component() {
5     // Вернет двухбуквенный код языка по стандарту ISO 639-1 строкой.
6     const lang = controller.currentLang;
7 }

1 # Python
2 from sbis import *
3 # Вернет строку в формате [язык], где
4 # [язык] – двухбуквенный код языка по стандарту ISO 639-1.
5 lang_str = CurrentLanguage()

1 // C++
2 #include <sbis-lib/forward/string.hpp>
3 #include <sbis-lib/utils/rk.hpp>
4
5 /**
6  * Вернет ISO code текущей локали в формате [язык], где
7  * [язык] – двухбуквенный код языка по стандарту ISO 639-1.
8  */
9 sbis::String lang = sbis::CurrentLanguage();
```

Получение текущей страны

```
1 // TypeScript
2 import {controller} from 'I18n/i18n';
3
4 export default function Component() {
5     // Вернет двухбуквенный код страны по стандарту ISO 3166-1 строкой.
6     const country = controller.currentCountry;
7 }

1 # Python
2 from sbis import *
3 # Вернет строку в формате [СТРАНА], где
4 # [СТРАНА] – двухбуквенный код страны по стандарту ISO 3166-1.
5 lang_str = CurrentCountry()

1 // C++
2 #include <sbis-lib/forward/string.hpp>
3 #include <sbis-lib/utils/rk.hpp>
4
5 /**
6  * Вернет ISO code текущей локали в формате [СТРАНА], где
7  * [СТРАНА] – двухбуквенный код страны по стандарту ISO 3166-1.
8  */
9 sbis::String lang = sbis::CurrentCountry();
```

Получение текущей локали

```
1 // TypeScript
2 import {controller} from 'I18n/i18n';
3
4 export default function Component() {
5     // Вернет строку в формате [язык]-[СТРАНА], где
6     // [язык] – двухбуквенный код языка по стандарту ISO 639-1.
7     // [СТРАНА] – двухбуквенный код страны по стандарту ISO 3166-1.
8     const lang = controller.currentLocale;
9 }
10
11 // C++
12 #include <sbis-lib/forward/string.hpp>
13 #include <sbis-lib/utils/rk.hpp>
14
15 /**
16 * Вернет ISO code текущей локали в формате [язык]-[СТРАНА], где
17 * [язык] – двухбуквенный код языка по стандарту ISO 639-1.
18 * [СТРАНА] – двухбуквенный код страны по стандарту ISO 3166-1.
19 */
20 sbis::String lang = sbis::CurrentLocale();
```

Задание контекста языка

```
1 # Python
2 from sbis import *
3 with LanguageContext("fr"):
4     // CurrentLanguage() == "fr"
```

Установка языкового контекста также работает при [межсервисном взаимодействии](#) с BLInvoke и FutureInvoke.

```
1 // C++
2 #include <sbis-lib/utils/language_context.hpp>
3 #include <sbis-lib/utils/rk.hpp>
4
5 void Func()
6 {
7     ScopeLanguageContext context( L"fr" );
8     // CurrentLanguage() == "fr"
9 }
```